Momentum on Delta-Neutral Straddles

OptionMetrics Formation $\rightarrow$ Next-Month Expiry Realization (No-Lookahead Pipeline)

Chris Liang

Icarus Fund Internship

February 2, 2026

Executive Summary

- **Research question:** Do underlyings with stronger *past straddle performance* continue to outperform in *next-month* straddle returns?
- **Strategy object:** monthly delta-neutral straddles built from OptionMetrics daily quotes (formation on 3rd Friday).
- **Core design principle: no lookahead** — rank on information available at month t and evaluate realized return in $t + 1$.
- **Robustness layers:**
  - Return capping (winsorization) to reduce domination by extreme option months
  - Inverse-volatility weighting to stabilize risk within buckets
  - Transaction cost / slippage model producing net-of-costs performance

Data Inputs & Key Fields

OptionMetrics daily option quotes (parquet per day)

- Path pattern: `BASE_DIR/{year}/optionmetrics_{YYYY-MM-DD}_nonzeroOI.parquet`
- Key columns: `date`, `exdate`, `secid`, `cp_flag`, `optionid`, `strike_price`, `best_bid`, `best_offer`, `open_interest`, `delta`, `expiry_indicator`
- Transformations:

$$K = \frac{\texttt{strike\_price}}{1000}, \quad \texttt{mid} = \frac{\texttt{best\_bid} + \texttt{best\_offer}}{2},$$

$$\texttt{spread} = \texttt{best\_offer} - \texttt{best\_bid}$$

CRSP-style mapping & prices

- `secid` $\rightarrow$ `permno` with validity windows `sdate..edate`
- Stock prices: `permno`, `date`, `prc`; use $S = |\texttt{prc}|$

Calendar Construction & Timing

- Formation date each month: **3rd Friday** (F_t)
- Target expiration: **3rd Friday of next month** (E_{t+1})
- Calendar table:
  - `cal[F_t, E_t1, month]`
  - `month` is the label for ranking month t

No-lookahead alignment:

- Build straddle using quotes on F_t
- Realize intrinsic payoff at E_{t+1} using stock price *at or before* E_{t+1} (asof merge)
- Return labeled `R_t1` corresponds to the realized performance for the $t \to t+1$ holding period

Step 1A: Calendar Construction (3rd Fridays) & Holding Period

Objective: define the monthly formation date and the next-month expiration date consistently.

- **Formation date:** F_t = **3rd Friday** of month t (e.g., 2018-01-19).
- **Target expiry:** E_{t+1} = **3rd Friday of month** $t+1$ (aligned one-to-one with F_t).
- Build a calendar table:
  - `cal[F_t, E_t1, month]`
  - `month` labels the ranking month t (e.g., `"2018-01"`).
- Sample range in your implementation: **2018-01** through **2023-12**.

Interpretation: Each row in `cal` defines a single holding period:

$$\text{enter on } F_t \quad \longrightarrow \quad \text{exit/realize on } E_{t+1}.$$

Step 1B: Data Inputs & Core Engineered Fields (Per Formation Day F_t)

Primary inputs (per day F_t):

- **OptionMetrics daily option quotes** (one parquet file per date)
- `secid` $\rightarrow$ `permno` **mapping** with validity windows `sdate..edate`
- **Underlying stock prices** (CRSP-style) used to compute S near expiry

For a given formation day F_t, compute engineered option fields:

$$\texttt{mid} = \frac{\texttt{best\_bid} + \texttt{best\_offer}}{2}, \qquad \texttt{spread} = \texttt{best\_offer} - \texttt{best\_bid},$$

$$K = \frac{\texttt{strike\_price}}{1000}.$$

- K is the strike in **dollars** (OptionMetrics stores `strike_price` scaled by 1000).
- These fields are computed for **every option row** on F_t before filtering.

Step 1C: Formation-Day Option Filtering (on F_t)

Goal: isolate the next-month expiry chain and ensure tradability/data quality.

Chain isolation:

- Keep "standard" options: `expiry_indicator` is blank/NaN
- Keep only options with `exdate == E_t1` (target expiry E_{t+1})

Required fields (drop rows with missing values):

- Quotes: `best_bid, best_offer, mid, spread`
- Contract + identifiers: `secid, optionid, cp_flag, strike_price` (and engineered K)
- Market/Greeks: `delta, open_interest`

Quote sanity constraints:

$$\texttt{best\_offer} \geq \texttt{best\_bid}, \qquad \texttt{best\_bid} > 0, \qquad \texttt{best\_offer} > 0, \qquad \texttt{mid} > 0.$$

Deliverable after Step 1: a clean, tradable option chain on F_t for contracts expiring at E_{t+1}.

Step 2A: Top-20 Liquidity Selection (Aggregate at the secidLevel)

Why "Top-20 first"? Your pipeline prioritizes liquid underlyings *before* pairing calls/puts to form straddles.

Compute liquidity components for each secidusing the filtered chain (Step 1 output):

$$\texttt{total\_oi}(\texttt{secid}) = \sum \texttt{open\_interest}, \qquad \texttt{med\_spread}(\texttt{secid}) = \text{median}(\texttt{spread}).$$

- total_oi proxies market attention / depth.
- med_spread proxies trading friction on that date and expiry.

Drop secidwith undefined liquidity inputs (e.g., missing median spread).

Step 2B: Liquidity Score & Ranking (Select the Top 20 secid)

Liquidity score definition:

$$\texttt{liq\_score} = \frac{\texttt{total\_oi}}{1 + \texttt{med\_spread}}.$$

- Larger open interest increases the score.
- Larger spreads decrease the score.
- The "+1" in the denominator prevents unstable behavior when spreads are very small.

Ranking rule:

- Sort secidby liq_score descending and keep the **top 20**.
- This selection is done **per month** (per formation day F_t).

Deliverable after Step 2B: a candidate list of 20 liquid secidon formation date F_t.

Step 2C: Enforce Valid secid→ permnoMapping at Formation Date F_t

Problem: secid-to-permnolinks can change over time, so mapping must be valid *at formation*.

Validity constraint (required):

$$\texttt{sdate} \leq F_t \leq \texttt{edate}.$$

- Drop candidate mappings that are not valid at F_t.
- Drop secidwith no valid permnoafter applying this rule.

Tie-breaking when multiple valid links exist for the same secid:

- Keep exactly one row per secid.
- Prefer the most recent valid mapping window (largest sdate).

Deliverable after Step 2C: Top-20 secid*with valid* permnoidentifiers attached.

Step 2D: Restrict the Option Chain to the Top-20 secidUniverse

Action: filter the full chain (Step 1 output) to retain only contracts whose secidis in the Top-20 list.

Resulting dataset properties:

- Still contains all strikes K and both call/put contracts for each selected secid.
- But now excludes illiquid tail secidthat would create unstable pairing and returns.

Deliverable after Step 2: a **Top-20-filtered chain** on F_t with expiry E_{t+1}, ready for straddle construction.

Step 3A: Build Call and Put Candidate Sets (Within Top-20 secid)

Calls:

- `cp_flag = 'C'`
- `open_interest > 0`
- **Delta filter:** $\Delta_C \in [0.25, 0.75]$

Puts:

- `cp_flag = 'P'`
- `open_interest > 0`

Column hygiene (conceptual):

- Keep only essential identifiers and pricing fields.
- Use distinct naming for call vs put fields to avoid collisions when pairing.

Step 3B: Pair Calls and Puts Into Same-Strike Straddles

Pairing rule (same-strike straddles): create call-put pairs by matching:

$$(\texttt{secid},\ \texttt{exdate},\ K,\ \texttt{strike\_price}).$$

- Same underlying `secid`
- Same expiration date (the target E_{t+1})
- Same strike (in dollars K)

Liquidity metric at the straddle level:

$$\texttt{combined\_spread} = \texttt{call\_spread} + \texttt{put\_spread}.$$

- Smaller `combined_spread` indicates a more tradable (cheaper-to-cross) straddle.

Step 3C: Delta-Neutral Convex Weights (Computation)

Goal: choose weights so the combined position is approximately delta-neutral at formation.

Stability guard: exclude pairs where the denominator is too small:

$$|\Delta_C - \Delta_P| \approx 0 \quad \Rightarrow \quad \text{unstable weight computation.}$$

Delta-neutral weight formulas:

$$w_C = \frac{-\Delta_P}{\Delta_C - \Delta_P}, \qquad w_P = 1 - w_C.$$

Economic constraint (convex combination):

$$w_C \in [0, 1] \quad \text{and} \quad w_P \in [0, 1].$$

- This excludes negative weights or leveraged combinations.
- Interpretable as allocating the straddle exposure across call and put legs.

Step 3C (cont.): Entry Premium Proxy (Formation Pricing)

Entry premium proxy (formation pricing):

$$\texttt{entry\_mid} = w_C \cdot \texttt{call\_mid} + w_P \cdot \texttt{put\_mid}.$$

Interpretation:

- `call_mid` and `put_mid` are midquotes on the formation date F_t.
- `entry_mid` is the premium paid for the delta-neutral straddle (pricing proxy).
- This value becomes the denominator when defining realized return in Step 4.

Sanity checks (conceptual):

- Require $\texttt{entry\_mid} > 0$.
- Carry forward (w_C, w_P) unchanged to expiry when valuing intrinsic payoff.

Step 3D: Select One "Best" Straddle per `secid`

Within each `secid`, choose exactly one straddle (one strike) to represent that underlying.

Selection rule:

- Primary: **minimize** `combined_spread`
- Tie-break: **minimize** `entry_mid`

Monthly output table (conceptual fields):

- Identifiers: `month`, `F_t`, `E_t1`, `secid`, `permno`
- Contract definition: strike K and the chosen call/put option IDs
- Formation state: deltas (Δ_C, Δ_P), weights (w_C, w_P)
- Pricing proxies: `call_mid`, `put_mid`, `entry_mid`, `combined_spread`

Deliverable after Step 3: a **selected** table with **up to 20 rows per month**.

Step 4A: Attach Underlying Stock Price at (or Before) Expiry E_{t+1}

Goal: for each selected row (`permno`, E_{t+1}), attach the underlying price S observed at or before E_{t+1}.

Underlying stock price construction:

- Stock data fields include `permno, date, prc`.
- Define the price level:

$$S = |\texttt{prc}|.$$

Expiry alignment rule (as-of match):

$$S(E_{t+1}) = \text{the most recent } S \text{ with stock\_date} \leq E_{t+1}, \quad \text{within the same } \texttt{permno}.$$

- This handles non-trading expiry dates cleanly (weekends/holidays): match to the last observed trading date.

Deliverable after Step 4A: each selected straddle row gains `stock_date` and S.

Step 4B: Intrinsic Payoffs at Expiry and Weighted Exit Value

Given strike K and expiry underlying price S:

Intrinsic payoffs:

$$\texttt{payoff}_C = \max(S - K, 0), \qquad \texttt{payoff}_P = \max(K - S, 0).$$

Exit value of the delta-neutral straddle (weights fixed at formation):

$$\texttt{exit\_intrinsic} = w_C \cdot \texttt{payoff}_C + w_P \cdot \texttt{payoff}_P.$$

Interpretation:

- You model the realized payoff at expiration using intrinsic value (exercise/expiry value).
- The same (w_C, w_P) chosen at formation is used to value the exit payoff.

Step 4C: Realized Straddle Return Over the Holding Period

Validity checks before computing returns:

$$\texttt{entry\_mid} > 0, \qquad S > 0, \qquad K > 0.$$

- Ensures denominator is valid and removes nonsensical price records.

Monthly realized return (formation $\rightarrow$ expiry):

$$\texttt{R\_t1} = \frac{\texttt{exit\_intrinsic} - \texttt{entry\_mid}}{\texttt{entry\_mid}}.$$

- `R_t1` is the realized return for the $t \rightarrow t+1$ holding period.
- This is the position-level performance measure used downstream for momentum sorting.

Step 4D: Monthly Output Table and Panel Construction

Monthly output: the "big" monthly routine returns a **selected** table that includes:

- Formation: `month`, F_t, E_{t+1}, `secid`, `permno`, strike K, `entry_mid`
- Expiry match: `stock_date`, S
- Payoffs: `payoff_c`, `payoff_p`, `exit_intrinsic`
- Realized return: `R_t1`

Across months:

- Loop over all `month` values in `cal`.
- Concatenate month-level **selected** tables into one panel dataset:

$$\texttt{all\_monthly} = \bigcup_t \texttt{selected}_t.$$

- If a formation-day option file is missing, the month is skipped (tracked in logs).

Deliverable after Step 4: a clean position-level panel `all_monthly` used for Step 5 momentum.

Step 5: Momentum Signal on Realized Straddle Returns (Overview)

Goal: convert each underlying's realized straddle return history into a *predictor* used for month-t sorting (no lookahead).

Inputs (from Step 4 output):

- Position-level / month-level panel `all_monthly` with at least:

`month`, `permno`, `R_t1`

- `R_t1` is the realized return from formation month t to next-month expiry $t+1$.

Output artifact:

- `step5`: a copy of `all_monthly` with an additional column `MOM_rt`.

Step 5A: Signal Definition and No-Lookahead Timing

Key timing rule: when constructing the signal at month t, exclude $t-1$ and avoid any lookahead.

Signal definition (per `permno`):

$$\mathrm{MOM}_{r,t} \;=\; \frac{1}{11}\sum_{k=2}^{12} R_{r,t-k} \quad \text{(mean of } R_{t-12},\ldots,R_{t-2}\text{)}$$

Interpretation:

- Uses a *lagged* 11-month window of realized straddle returns.
- Skips $t-1$ on purpose (guardrail against short-horizon microstructure and timing leakage).
- Produces a cross-sectional ranking signal available *at formation month* t.

Step 5B: Rolling Window Mechanics and Data Availability Rule

Rolling construction idea: for each `permno`, take the time series of `R_t1`, shift by 2 months, then average the most recent 11 values.

Stability / minimum-history rule:

- If fewer than **8 non-missing** returns exist inside the 11-month window,

  `MOM_rt` is set to NaN.

- This prevents the signal from being driven by very short or sparse histories.

Practical consequence:

- Early months for each `permno`will not have `MOM_rt`.
- Later steps will naturally drop rows missing `MOM_rt` (and/or `R_next`).

Step 5C: Step-5 Deliverables for Diagnostics

Deliverables you generate from `step5`:

- `mom_table`: month-by-month (or overall) table of `MOM_rt` availability and values by `permno`.
- `mom_table_stats`: summary statistics overview (counts, mean/median, dispersion).
- **Heatmap diagnostic:** visualize `MOM_rt` coverage over time using
  - only the **top 60** `permno` with the most non-missing `MOM_rt` observations, and
  - months on the x-axis, `permno`on the y-axis, color = `MOM_rt`.

Purpose: confirm signal coverage patterns and ensure the NaN rule behaves as intended.

Step 6: Quintile Sorting and Next-Month Evaluation (Overview)

Goal: at each month t, sort underlyings by `MOM_rt` and evaluate performance using the *next-month* realized return.

Core no-lookahead alignment:

- **Signal month:** t uses `MOM_rt` computed from returns up to $t-2$.
- **Evaluation month:** measure outcome using the next realized return

$$R_{\text{next}} \equiv R_{t+1}.$$

Working panel:

- Create `panel6` containing:

`month`, `permno`, `R_t1`, `MOM_rt`

- Sort by `permnothen` by `month` to ensure time ordering is correct.

Step 6A: Constructing Next-Month Return R_next and Dropping Missing Rows

Next-month realized return:

$$\texttt{R\_next} = R_{t+1}$$

constructed by shifting the realized return series forward by one month *within each* permno.

Automatic row losses (expected):

- **Early months drop** because MOM_rt is undefined until enough history accumulates.
- **Final month drops** because it has no R_next.
- Any row missing **any** of:

  month, permno, MOM_rt, R_next

  is removed from the sorting/evaluation sample.

Result: a clean month-by-month cross-section ready for bucketing.

Step 6B: Outlier Control via Winsorization (Return Capping Guardrail)

Why cap? A single extreme options month can dominate both momentum signals and portfolio averages.

Capping bounds (modeling guardrail):

$$\texttt{CAP\_LO} = -0.95, \qquad \texttt{CAP\_HI} = 5.0$$

$$R^{cap} = \mathrm{clip}(R,\ \texttt{CAP\_LO},\ \texttt{CAP\_HI})$$

Implementation concept:

- Keep raw `R_next` unchanged for transparency.
- Create `R_next_cap` used for aggregation and portfolio construction.
- Interpretation: returns below -95% are treated as -95%, and returns above $+500\%$ are treated as $+500\%$.

Step 6C: Bucket Formation and Long–Short Portfolio Construction

Set the number of quantile buckets:

$$Q = 5 \quad \text{(quintiles)}$$

Bucket assignment rule (within each month):

- Sort (or rank) underlyings by `MOM_rt`.
- Assign buckets $1, \ldots, Q$ using quantile cutoffs.
- If a month has insufficient valid observations to form Q buckets, assign **NA** (skip that month for LS).

Portfolio return proxies (using capped next-month returns):

$$\bar{R}_{q,t+1} = \text{mean of } R^{cap}_{i,t+1} \text{ for names in bucket } q$$

$$\texttt{LS\_spread}_t = \bar{R}_{Q,t+1} - \bar{R}_{1,t+1}, \qquad \texttt{LS\_5050}_t = 0.5\left(\bar{R}_{Q,t+1} - \bar{R}_{1,t+1}\right)$$

Step 7: Performance Evaluation for Long–Short Series (Overview)

Goal: summarize and visualize the time-series performance of:

`LS_5050` and `LS_spread`.

Evaluation function (conceptual): `eval_series(series)` returns a dictionary of performance statistics.

Core monthly statistics (sample estimates):

- Sample mean: $\bar{R}$
- Sample standard deviation: s
- Monthly Sharpe:

$$\text{Sharpe}_m = \frac{\bar{R}}{s}$$

Step 7A: Inference Proxy and Annualization (IID Approximation)

Let T be the number of monthly observations in the long–short series.

t-statistic on mean return:

$$t = \frac{\bar{R}}{s/\sqrt{T}}$$

Annualization (IID approximation):

$$\mu_{ann} = 12\,\bar{R}, \qquad \sigma_{ann} = \sqrt{12}\,s, \qquad \text{Sharpe}_{ann} = \frac{\mu_{ann}}{\sigma_{ann}}.$$

Note: This is a reporting convention; serial correlation and regime effects may motivate HAC/Newey–West later.

Step 7B: Visualization Rules (Additive vs Compounded) and Histogram

Additive cumulative P&L-style curve (always defined):

$$\texttt{cum\_add}(t) = \sum_{\tau \le t} R_\tau.$$

- Safe even if some month has return $\le -100\%$.
- Used as the primary cumulative visualization in this project.

Compounded cumulative return (only if feasible):

$$\texttt{cum\_mult}(t) = \prod_{\tau \le t} (1 + R_\tau) - 1,$$

- Only plot if **no** month satisfies $R_\tau \le -1$.
- Interpretation: a month $\le -100\%$ implies theoretical capital wipeout.

Distribution plot:

- Histogram of monthly returns (x-axis: monthly return, y-axis: count).

Step 7C: Reporting Checklist (What You Produce)

Apply the same evaluation to both long–short series:

`LS_5050` and `LS_spread`.

For each series, report:

- Summary dictionary/table from `eval_series` (mean, vol, Sharpe, t-stat, annualized metrics)
- Additive cumulative plot
- (Optional) compounded plot when feasible
- Histogram of monthly returns

Suggested slide insert: replace placeholders with your generated plots (PNG/PDF).

Why can_compound Can Be False (When $R \leq -1$)

Definition of compounded cumulative return:

$$\texttt{cum\_mult}(t) = \prod_{\tau \leq t} (1 + R_\tau) - 1.$$

Feasibility condition:

$\texttt{cum\_mult}(t)$ is well-defined as a "wealth process" only if $1 + R_\tau > 0 \ \forall \tau$,

equivalently,

$$R_\tau > -1 \quad \text{for every month } \tau.$$

Why `can_compound` Can Be False (When $R \leq -1$) — Continued

Interpretation:

- $R_\tau \leq -1$ means a **loss of 100% or more** in that month.
- A month with $R_\tau = -1$ implies the portfolio wealth hits **zero**.
- A month with $R_\tau < -1$ implies the model would require **negative wealth / borrowing**, so compounding no longer matches a self-financing wealth curve.

Therefore: `can_compound = False` is a safety flag whenever $\min_\tau R_\tau \leq -1$.

How a Long–Short Series Can Produce $R < -1$ (Even Without Leverage in Code)

Your long–short series is a return spread (difference of bucket returns):

$$\texttt{LS\_spread}_t = \bar{R}_{Q,t} - \bar{R}_{1,t}, \qquad \texttt{LS\_5050}_t = \tfrac{1}{2}\left(\bar{R}_{Q,t} - \bar{R}_{1,t}\right).$$

Key point: `LS_spread` is not automatically a "wealth return" unless you explicitly define a self-financing allocation and a wealth update rule.

Why $R < -1$ can happen:

- The difference of two returns can be smaller than -1:

$$\bar{R}_{Q,t} - \bar{R}_{1,t} < -1 \quad \text{if losers outperform winners by more than } 100\%.$$

- This can be amplified in option returns because **straddle returns can be very large in magnitude** when:
  - entry premium (`entry_mid`) is small, so $\frac{\texttt{exit}-\texttt{entry}}{\texttt{entry}}$ can be extreme, and/or
  - tail events create very large intrinsic payoffs for some names while others decay.
- In such months, `LS_spread` is better viewed as a **P&L-like spread series** rather than a pure multiplicative return.

Step 8: Risk Management via Inverse-Volatility Weighting (Overview)

Goal: keep bucket membership fixed (from Step 6), but allocate within each bucket by inverse risk.

High-level idea:

- **Same sorting signal** (`MOM_rt`) and same bucket labels.
- Replace equal-weight bucket means with **inverse-vol weighted** means.
- Use the *same lag structure* (no lookahead) for volatility estimates.

Step 8A: Per-Underlying Risk Estimate (No-Lookahead, Same Window)

Per-permno volatility estimate at month t:

$$\sigma_{r,t} \;=\; \mathrm{std}(R_{r,t-12}, \ldots, R_{r,t-2})$$

- Uses the same 11-month lag window as momentum.
- Excludes $t-1$ and avoids lookahead.
- Requires a minimum history (e.g., at least 8 non-missing returns) for stability.

Numerical guard: add a small ε to avoid division by zero.

Step 8B: Inverse-Vol Weights and Bucket Return Construction

Raw inverse-vol weight:

$$w_{r,t}^{raw} = \frac{1}{\sigma_{r,t} + \varepsilon}$$

Normalize within each (month, bucket):

$$w_{r,t} = \frac{w_{r,t}^{raw}}{\sum_{i \in \text{bucket}} w_{i,t}^{raw}}$$

Inverse-vol bucket return (use capped next-month returns):

$$\bar{R}_{q,t+1}^{iv} = \sum_{r \in q} w_{r,t}\, R_{r,t+1}^{cap}$$

Inverse-vol long–short series:

$$\texttt{LS\_5050\_iv}_t = 0.5\left(\bar{R}_{Q,t+1}^{iv} - \bar{R}_{1,t+1}^{iv}\right).$$

Step 8C: Evaluation Mirrors Step 7 (Now on Inverse-Vol Series)

Repeat Step 7 evaluation for inverse-vol results:

- `eval_series(LS_5050_iv)` summary table
- Additive cumulative curve for `LS_5050_iv`
- Histogram of monthly `LS_5050_iv` returns

Purpose: verify whether inverse-vol weighting stabilizes the long–short distribution and improves risk-adjusted performance.

Step 9: Transaction Costs & Slippage (Net-of-Costs Pipeline Overview)

Goal: adjust position returns for realistic execution frictions, then rerun Steps 5–8 on net returns.

Cost components (position-level):

- **Entry slippage rate** s_{entry}: you pay above the mid on entry.
- **Exit cost / haircut rate** s_{exit}: you receive below intrinsic on exit.
- **Fixed bps trading cost per leg** (commissions/fees/spread-impact proxy).
- **Number of legs:** a straddle has two option legs (call + put).

Output artifact: `R_t1_net` for each position/month.

Step 9A: Effective Entry/Exit Values Under Costs

Effective entry premium: pay above mid

$$\texttt{entry\_eff} = \texttt{entry\_mid} \cdot (1 + s_{entry}).$$

Effective exit value: receive below intrinsic

$$\texttt{exit\_eff} = \texttt{exit\_intrinsic} \cdot (1 - s_{exit}).$$

Bps-based cost rate (per position):

$$\texttt{cost\_rate\_total} = \frac{\texttt{cost\_bps\_per\_leg}}{10000} \times \#\texttt{legs}.$$

- Apply the bps rate to a notional proxy (commonly the premium scale, e.g., `entry_mid`).

Step 9B: Net Return Definition (Net-of-Costs R_t1_net)

Round-trip cost proxy (premium-scaled):

$$\texttt{roundtrip\_cost} = \texttt{entry\_mid} \cdot \texttt{cost\_rate\_total}.$$

Net return definition:

$$\texttt{R\_t1\_net} = \frac{\texttt{exit\_eff} - \texttt{entry\_eff} - \texttt{roundtrip\_cost}}{\texttt{entry\_mid}}.$$

Parameterization knobs:

- s_{entry} (entry slippage), s_{exit} (exit haircut)
- cost_bps_per_leg and #legs (typically 2)

Step 9C: Rerun Steps 5–8 on Net Returns (Gross vs Net Comparison)

Pipeline rerun on net returns:

- Replace `R_t1` with `R_t1_net`.
- Recompute:

$$\texttt{MOM\_rt\_net}, \quad \texttt{R\_next\_net}, \quad \texttt{R\_next\_net\_cap}$$

- Repeat Step 6 bucketing and compute:

$$\texttt{LS\_5050\_net}, \quad \texttt{LS\_spread\_net}.$$

- Repeat Step 8 inverse-vol weighting on net returns:

$$\texttt{LS\_5050\_iv\_net} \quad \text{(if you implement the net+iv combination).}$$

Deliverables: side-by-side summary tables and cumulative plots for **gross vs net**.

Implementation Architecture (Pipeline Map)

High-level dataflow (monthly, no-lookahead):

- Calendar: `cal[month, F_t, E_t1]` defines formation F_t and expiry E_{t+1} per month.
- For each `month`:
  - Load OptionMetrics chain on F_t and filter to expiry E_{t+1}
  - Select Top-20 `secid`by `liq_score`
  - Build delta-neutral straddles and select 1 per `secid`$\Rightarrow$ `selected`
  - Attach underlying price $S(E_{t+1})$ via backward as-of match $\Rightarrow$ compute `R_t1`
- Stack month-level outputs:

$$\texttt{all\_monthly} = \bigcup_t \texttt{selected}_t$$

- Cross-sectional strategy steps: `all_monthly` $\rightarrow$ `step5` $\rightarrow$ `panel6/membership` $\rightarrow$ LS series

Robustness variants: rerun the same pipeline using (i) inverse-vol weighting and (ii) net-of-cost returns.

Implementation Architecture (Key Functions / Artifacts)

Core functions (conceptual roles):

- `third_fridays()` → build `cal[F_t, E_t1, month]`
- `load_opt_day(F_t)` → daily chain with engineered `mid, spread, K`
- `month_return_top20first(month)` → monthly `selected` with `R_t1`
- `attach_stock_price_at_expiry()` → backward as-of match by `permno`
- `bucketize_mom()` → month-t quantile assignment (handles insufficient names)
- `eval_series()`, `plot_cumulative()`, `plot_hist()` → reporting layer

Main dataframes (named as in notebook):

- `all_monthly`: stacked monthly selected straddles + `R_t1`
- `step5`: adds `MOM_rt` and diagnostics (`mom_table`, `mom_table_stats`)
- `panel6`: adds `R_next`, `R_next_cap`, and bucket labels
- `LS_5050`, `LS_spread`: equal-weight long–short series
- `LS_5050_iv`: inverse-vol long–short series
- `R_t1_net`, `LS_5050_net`: net-of-costs pipeline outputs

Validation & Controls (Data Quality + No-Lookahead)

Formation-day option sanity checks:

- Non-crossed markets: best_offer $\geq$ best_bid; best_bid, best_offer, mid > 0
- Liquidity screen: open_interest > 0; Top-20 secidby liq_score
- Standard contracts only: expiry_indicator blank/NaN; filter to exdate == E_t1

Mapping & timing integrity:

- Mapping validity at formation: sdate $\leq F_t \leq$ edate (tie-break by most recent sdate)
- No-lookahead signal: shift(2) and window t-12 to t-2
- No-lookahead evaluation: R_next constructed within-permnoby next-month shift
- Expiry price alignment: backward as-of match ensures stock_date $\leq E_{t+1}$

Validation & Controls (Guardrails + Interpretation)

Weight stability guards (construction):

- Exclude unstable pairs when $|\Delta_C - \Delta_P|$ is too small
- Enforce convex weights: $w_C, w_P \in [0, 1]$
- Require `entry_mid` > 0, $S > 0$, $K > 0$

Outlier control for aggregation (report both):

$$R^{cap} = \mathrm{clip}(R, -0.95, 5.0)$$

- Used for bucket means and long–short series stability
- Results can be shown **capped** and **uncapped** for transparency

Compounding feasibility check:

- Only plot compounded $\prod(1 + R) - 1$ if $\min(R) > -1$ (`can_compound`)
- Otherwise, use additive cumulative $\sum R$ (always defined)

Step 6 (Strategy 2): Top-3 / Bottom-3 vs Quintiles (What Changes?)

Original Step 6 (Quintiles):

- For each month t, assign all valid permnointo $Q = 5$ buckets using qcut on MOM_rt.
- Compute bucket means $\bar{R}_{1,t+1}$ and $\bar{R}_{Q,t+1}$ using next-month returns.
- Long–short:

$$\texttt{LS\_spread}_t = \bar{R}_{Q,t+1} - \bar{R}_{1,t+1}, \qquad \texttt{LS\_5050}_t = \tfrac{1}{2}(\bar{R}_{Q,t+1} - \bar{R}_{1,t+1})$$

Strategy 2 (Top/Bottom 3):

- For each month t, **only keep extremes:** top 3 names (highest MOM_rt) and bottom 3 names (lowest MOM_rt).
- Compute **long mean** and **short mean** using those 3+3 names only.
- Logic: concentrate exposure on the strongest signal tails to potentially increase the spread.

Step 6 (Strategy 2): Selection Rule (Ranking + Fallback)

Inputs for month t: `month, permno, MOM_rt, R_next_cap` (after Step 5).

Ranking (no lookahead):

- Sort by `MOM_rt` **descending** $\Rightarrow$ candidate long list.
- Sort by `MOM_rt` **ascending** $\Rightarrow$ candidate short list.

Top-3 / Bottom-3 selection with fallback:

- **Longs:** walk down the ranked list and take the first $N_{\text{LONG}} = 3$ names with `R_next_cap` not missing.
- **Shorts:** walk up from the bottom and take the first $N_{\text{SHORT}} = 3$ names with `R_next_cap` not missing.
- If a name appears in both sets (tiny universe edge case), drop overlaps from shorts and **refill** from the next available short candidates.

Membership table: store selected names with labels `side` $\in$ {`LONG, SHORT`} for Step 7 reporting.

Step 6 (Strategy 2): Portfolio Returns (Means of Extremes) + Guardrails

Next-month outcome variable:

$$\texttt{R\_next} = R_{t+1} \quad \text{(constructed within each permno)}.$$

Winsorization / capping (portfolio mechanics):

$$\texttt{CAP\_LO} = -0.95, \qquad \texttt{CAP\_HI} = 5.0, \qquad \texttt{R\_next\_cap} = \mathrm{clip}(\texttt{R\_next}, \texttt{CAP\_LO}, \texttt{CAP\_HI}).$$

Month-t long and short means (using selected 3+3 names):

$$\texttt{long\_ret\_next}_t = \frac{1}{N_L} \sum_{i \in \mathcal{L}_t} \texttt{R\_next\_cap}_{i,t}, \qquad \texttt{short\_ret\_next}_t = \frac{1}{N_S} \sum_{j \in \mathcal{S}_t} \texttt{R\_next\_cap}_{j,t}.$$

Long–short series:

$$\texttt{LS\_spread}_t = \texttt{long\_ret\_next}_t - \texttt{short\_ret\_next}_t,$$

$$\texttt{LS\_5050}_t = \tfrac{1}{2}(\texttt{long\_ret\_next}_t - \texttt{short\_ret\_next}_t).$$

Diagnostics captured per month: `n_long` and `n_short` (can be < 3 if too many missing outcomes).

Strategy 3: Adding Early Exits (Stop-Loss + Profit-Take) to Strategy 2

Baseline (Strategy 2):

- Select **Top-3** and **Bottom-3** each month using `MOM_rt`.
- Hold each selected straddle from formation F_t all the way to expiry E_{t+1}.
- Realized return uses intrinsic payoff at E_{t+1}:

$$\texttt{R\_t1} = \frac{\texttt{exit\_intrinsic} - \texttt{entry\_mid}}{\texttt{entry\_mid}}.$$

New feature (Strategy 3):

- Keep **the same selection logic** (Top-3/Bottom-3 by momentum).
- Change the **holding-period exit rule**:
  - allow an **early exit** if the straddle value hits a stop-loss or profit-take level,
  - otherwise, still hold to expiry as before.
- New realized return variable: `R_t1_enh`.

Strategy 3: Daily Monitoring Signal (Combined Straddle Mid Value)

Key idea: after entering on F_t, track the *mark-to-market* value using daily midquotes.

For each selected straddle (fixed weights from formation):

$$\texttt{combined\_mid}(d) = w_C \cdot \texttt{call\_mid}(d) + w_P \cdot \texttt{put\_mid}(d), \qquad d \in (F_t, E_{t+1}].$$

Notes:

- (w_C, w_P) are the **same delta-neutral weights** computed at formation (Step 3).
- `call_mid`(d), `put_mid`(d) come from OptionMetrics daily quotes on business days.
- We begin checking on the **first business day after** F_t to avoid using the same-day entry quote as an exit.

Strategy 3: Early Exit Triggers (Stop-Loss and Profit-Take)

Let `entry_mid` be the formation-day premium proxy.

Stop-loss threshold (drawdown control):

$$\texttt{stop\_level} = (1 - \alpha) \cdot \texttt{entry\_mid}, \qquad \alpha = 0.75$$

$$\text{STOP if } \texttt{combined\_mid}(d) \leq \texttt{stop\_level}.$$

Profit-take threshold (profit booking):

$$\texttt{profit\_level} = (1 + \beta) \cdot \texttt{entry\_mid}, \qquad \beta = 0.80$$

$$\text{PROFIT if } \texttt{combined\_mid}(d) \geq \texttt{profit\_level}.$$

First-hit logic: scan dates in time order; exit at the **first** day a trigger is hit.

Strategy 3: Exit Value and Enhanced Return Definition (R_t1_enh)

If an early exit occurs at date $d^* \in (F_t, E_{t+1}]$:

$$\texttt{exit\_value} = \texttt{combined\_mid}(d^*)$$

$$\texttt{R\_t1\_enh} = \frac{\texttt{exit\_value} - \texttt{entry\_mid}}{\texttt{entry\_mid}}.$$

If no trigger is hit (HOLD to expiry):

- Use the original intrinsic-at-expiry exit (same as Strategy 2):

$$\texttt{R\_t1\_enh} \equiv \texttt{R\_t1}.$$

Exit bookkeeping fields (for reporting):

- exit_reason $\in \{$STOP, PROFIT, HOLD$\}$
- exit_date and exit_value (only meaningful for early exits)

Strategy 3: Implementation Mechanics (Daily Scan, Minimal Critical Lines)

Daily scan set:

- Evaluate only business days:

$$d \in \{F_t + 1\text{B},\ F_t + 2\text{B},\ \ldots,\ E_{t+1}\}$$

- Require both leg mids available on day d; otherwise skip that day for that position.

Two critical lines (conceptual implementation):

- Combined mark-to-market value:

$$\texttt{combined} = w_C \cdot \texttt{call\_mid}(d) + w_P \cdot \texttt{put\_mid}(d)$$

- Trigger checks (first-hit):

$$\texttt{combined} \leq \texttt{stop\_level} \Rightarrow \texttt{STOP}, \qquad \texttt{combined} \geq \texttt{profit\_level} \Rightarrow \texttt{PROFIT}.$$

Downstream pipeline:

- Replace `R_t1` with `R_t1_enh` before Step 5 momentum and Step 6 Top-3/Bottom-3 evaluation.

Five Historically Useful Improvements (Current vs Better Alternatives)

Goal: identify the *highest-impact* methodological upgrades that historically matter most for option strategies:

1. **Liquidity / universe definition:** what does "tradable" really mean?
2. **Strike (straddle) selection:** choose the representative straddle per `secid` more robustly.
3. **Return measurement at exit:** intrinsic-at-expiry vs mark-to-market / settlement-aware valuation.
4. **Portfolio return definition:** spread differences vs a true self-financing long–short return.
5. **Execution costs:** constant slippage vs liquidity-adaptive cost models.

Each item below states:

(i) Current treatment $\Rightarrow$ **(ii) Alternative treatment** $\Rightarrow$ **why it may be better**.

(1) Liquidity / Universe Definition: What Does "Top-20" Mean?

Current treatment (your pipeline):

- On formation day F_t, after filtering to expiry E_{t+1}, compute per-secid:

$$\texttt{total\_oi} = \sum \texttt{open\_interest}, \qquad \texttt{med\_spread} = \text{median}(\texttt{spread})$$

$$\texttt{liq\_score} = \frac{\texttt{total\_oi}}{1 + \texttt{med\_spread}}$$

- Rank secidby liq_score and keep **Top 20**.

Alternative treatment (often more robust historically):

- Replace absolute spread with a **relative** friction proxy:

$$\texttt{rel\_spread} = \frac{\texttt{spread}}{\texttt{mid}} \quad \text{or} \quad \frac{\texttt{spread}}{S}$$

- Use an **activity-weighted** spread (OI-weighted):

$$\texttt{w\_spread} = \frac{\sum_i \texttt{OI}_i \cdot \texttt{spread}_i}{\sum_i \texttt{OI}_i}$$

- Score example:

$$\texttt{liq\_score}^{alt} = \frac{\texttt{total\_oi}}{1 + \texttt{w\_rel\_spread}}$$

(1) Liquidity / Universe: Why the Alternative Can Be Better

Why the current approach can be suboptimal:

- **Absolute** spreads are not comparable across underlyings with different option price levels.
- Median spread treats inactive strikes equally as active ones (can over-weight "dead" strikes).
- Total OI across the chain may be large even if the **ATM region** is not liquid.

Why the alternative often improves performance and stability:

- Relative spreads better approximate proportional trading friction.
- OI-weighted spreads align the liquidity metric with where trading actually concentrates.
- Historically reduces universe churn and improves out-of-sample stability for cross-sectional signals.

Tradeoff: slightly more engineering + stronger assumptions about which strikes matter most (usually a feature, not a bug).

(2) Selecting the "Best" Straddle per secid

Current treatment:

- Pair by same secid, same E_{t+1}, same strike K.
- Compute combined_spread = call_spread + put_spread.
- Pick one pair per secid:
  - Primary: minimize combined_spread
  - Tie-break: minimize entry_mid
- Pair-level depth (call_oi, put_oi) not directly used.

Alternative (often more robust):

- Depth proxy: pair_oi = min(call_oi, put_oi).
- Tradability score:

$$\text{score} = \frac{\text{pair\_oi}}{1 + \text{rel\_combined\_spread}}$$

- Premium floor: entry_mid $\geq$ min_premium to avoid tiny denominators.

(2) Straddle Selection: Why This Matters for Returns and Outliers

Why the current approach can distort results:

- Minimizing spread alone can select strikes with **low actual tradable depth**.
- A very small `entry_mid` can mechanically create extreme returns:

$$\texttt{R\_t1} = \frac{\texttt{exit} - \texttt{entry\_mid}}{\texttt{entry\_mid}} \quad \Rightarrow \quad \texttt{entry\_mid} \downarrow \Rightarrow \ |\texttt{R\_t1}| \uparrow$$

- That can increase tail risk and create reliance on winsorization in Step 6.

Why the alternative often improves robustness:

- Pair-level OI enforces that *both legs* are meaningfully tradable.
- Relative-spread scoring makes selection comparable across names.
- Premium floors reduce unstable return denominators (fewer artificial outliers).

(3) Exit Valuation: Intrinsic-at-Expiry vs Tradable Mark-to-Market

Current treatment (your pipeline):

- Attach underlying price $S(E_{t+1})$ (last observed stock price with date $\leq E_{t+1}$).
- Value the straddle at expiry using **intrinsic payoff**:

$$\texttt{payoff}_C = \max(S - K, 0), \qquad \texttt{payoff}_P = \max(K - S, 0)$$

$$\texttt{exit\_intrinsic} = w_C \cdot \texttt{payoff}_C + w_P \cdot \texttt{payoff}_P$$

Alternative treatment (often closer to tradable P&L historically):

- Use a **mark-to-market exit** near E_{t+1} using option midquotes:

$$\texttt{exit\_mid} = w_C \cdot \texttt{call\_mid}(E_{t+1}) + w_P \cdot \texttt{put\_mid}(E_{t+1})$$

- Or use **settlement-aware** pricing conventions if available (AM/PM settlement alignment).

(3) Exit Valuation: Why the Alternative Can Change Conclusions

Why intrinsic-only can be misleading:

- Ignores **tradability** at exit (you cannot usually transact at intrinsic).
- If closing before final settlement, the option can still have time value; intrinsic-only can under/overstate P&L.
- Strategy comparisons can shift if some names systematically retain more extrinsic value near expiry.

Why mark-to-market / settlement-aware exit is often better:

- Better aligns with realistic execution (`mid` or `mid` $\pm$ half-spread).
- Historically reduces "paper alpha" caused by valuation conventions.

Tradeoff: requires option quotes at/near E_{t+1} and careful handling of missing quotes / late trading days.

(4) Portfolio Return: Difference of Bucket Means vs Self-Financing Long–Short

Current treatment (your pipeline):

- Compute bucket average returns (equal-weight) using capped next-month returns:

$$\bar{R}_{q,t+1} = \text{mean}\left(R^{cap}_{i,t+1}\right)$$

- Define:

$$\texttt{LS\_spread}_t = \bar{R}_{Q,t+1} - \bar{R}_{1,t+1} \qquad \texttt{LS\_5050}_t = 0.5(\bar{R}_{Q,t+1} - \bar{R}_{1,t+1})$$

Alternative treatment (historically crucial for interpretability):

- Define an explicit **self-financing** long–short portfolio return:

$$R^{LS}_{t+1} = \sum_{i \in Q} w^L_{i,t} R_{i,t+1} - \sum_{j \in 1} w^S_{j,t} R_{j,t+1}$$

- Choose weights so net capital is defined (e.g., dollar-neutral with a leverage convention).

(4) Portfolio Return: Why This Impacts Compounding and Risk Interpretation

Why the current approach can confuse compounding:

- A *difference of average returns* is not automatically the return on a tradable wealth process.
- This is exactly why `can_compound` can fail for `LS_spread` in your diagnostics.

Why the alternative is better historically:

- Produces a return series with a clear investment interpretation (what a portfolio would earn).
- Enables consistent performance metrics (compounded equity curve, drawdowns) without ambiguity.
- Makes cost modeling more coherent (turnover and dollar exposure are defined).

Tradeoff: requires a declared leverage/notional convention and shorting assumptions (standard in academic factors).

(5) Transaction Costs: Constant Parameters vs Liquidity-Adaptive Costs

Current treatment (your Step 9):

- Entry slippage: $\texttt{entry\_eff} = \texttt{entry\_mid}(1 + s_{entry})$
- Exit haircut: $\texttt{exit\_eff} = \texttt{exit\_intrinsic}(1 - s_{exit})$
- Bps cost: $\texttt{cost\_rate\_total} = \frac{\texttt{cost\_bps\_per\_leg}}{10000} \times \#\texttt{legs}$
- Net return:

$$\texttt{R\_t1\_net} = \frac{\texttt{exit\_eff} - \texttt{entry\_eff} - \texttt{roundtrip\_cost}}{\texttt{entry\_mid}}$$

Alternative treatment (commonly more realistic historically):

- Make costs **state-dependent** on spread / liquidity:

$$s_{entry} = a \cdot \texttt{rel\_spread}, \qquad s_{exit} = b \cdot \texttt{rel\_spread}$$

- Or a half-spread execution rule:

$$\texttt{entry\_eff} = \texttt{entry\_mid} + \tfrac{1}{2}\texttt{combined\_spread},$$

$$\texttt{exit\_eff} = \texttt{exit\_mid} - \tfrac{1}{2}\texttt{combined\_spread}$$

(5) Costs: Why Liquidity-Adaptive Modeling Often Changes Net Alpha

Why constant costs can be misleading:

- Understates costs exactly when spreads widen (stress regimes), overstating strategy resilience.
- Makes cross-sectional comparisons unfair: illiquid names get the same cost assumption as liquid names.

Why adaptive costs are often better historically:

- Net performance becomes consistent with observed microstructure: wide spreads $\Rightarrow$ larger slippage.
- Helps detect whether alpha is *real* or simply compensation for liquidity provision / execution risk.

Tradeoff: adds model risk (choose parameters a, b), so results should be shown across a grid of cost assumptions.

Next Steps (If Time Allows)

Methodology upgrades:

- **Calendar robustness:** handle holiday shifts explicitly; confirm expiry conventions match listed monthly expiries
- **Mark-to-market P&L:** use midquotes over holding period (daily) vs intrinsic-only at expiry
- **Delta re-hedging:** quantify impact of hedge frequency on realized returns
- **Alternative constructions:** ATM straddles vs delta-neutral convex weights; different delta bands

Inference & attribution:

- Newey–West/HAC t-stats for serial correlation; regime splits (e.g., volatility regimes)
- Risk attribution: exposures to implied vol level/skew and underlying characteristics
- Sensitivity: cap bounds, cost parameters, and Top-N liquidity cutoffs